AI ROUTE EXPLORER

Intelligent Tourist Discovery and Route Optimization System

PROJECT VISION
Discover and rank tourist attractions along a travel route, personalize suggestions to traveler interests, and minimize added distance and travel time.

Project dimension	Definition
Type	AI + geospatial intelligence + recommendation system
Primary output	Route-aware attraction recommendations with detour estimates
Implementation	Open-source first, without AWS services
Target roles	AI Engineer, Applied AI Engineer, ML Engineer, Backend AI Engineer

PORTFOLIO PROJECT SPECIFICATION

1. Executive Summary

AI Route Explorer is an intelligent road-trip planning system that discovers tourist attractions along a user's route, estimates the additional travel required for each stop, and ranks the best options based on traveler interests. Unlike ordinary nearby-search experiences, the system is route-aware: a recommendation must be relevant and reasonably accessible from the actual path between source and destination.

CORE VALUE PROPOSITION
Turn a basic A-to-B route into a personalized exploration plan without forcing the traveler to search for each attraction manually.

Example use case

Input	Example
Source	Hyderabad
Destination	Araku
Interests	Nature, photography
Maximum detour	10 km or a user-selected time limit

Expected result

- Recommended attractions ordered by suitability
- Distance from the original route
- Estimated additional travel time
- Updated route with selected stops
- Interactive map visualization

Why this project matters

The project combines backend engineering, geospatial data processing, recommendation logic, route optimization, LLM integration, and future machine-learning personalization in one cohesive product. It is designed to progress from a working non-AI MVP to a production-style intelligent system.

2. Problem Statement and Goals

Current traveler pain points

- Navigation tools prioritize reaching the destination, not discovering attractions along the way.
- Travelers must search, compare, and add each attraction manually.
- Nearby attractions can create large detours because proximity to the traveler does not guarantee proximity to the route.
- Generic suggestions may not match interests such as nature, history, photography, or adventure.
- Manual planning makes it easy to miss worthwhile stops.

Proposed solution

1. Accept source, destination, interests, and detour constraints.
2. Generate the primary route and its geometry.
3. Discover candidate attractions around the route corridor.
4. Estimate route deviation, added distance, and added time.
5. Filter and rank attractions using transparent scoring.
6. Present the recommended stops on a map and optionally rebuild the route.

Success criteria for the MVP

Metric	Target behavior
Route correctness	A valid route is produced for supported source and destination locations.
Route relevance	Candidates are discovered near the route corridor, not merely near the destination.
Detour transparency	Each suggestion includes added distance and estimated time.
Usability	The user can understand why a stop was suggested.
Reliability	Invalid input, no-route cases, and empty attraction results are handled clearly.

3. MVP Scope and Boundaries

FIRST RELEASE PRINCIPLE
Build the geospatial core before adding agents or machine learning. A reliable route and detour engine is the foundation of every later AI feature.

Included in MVP

- Source and destination input
- Driving route retrieval
- Route geometry extraction
- Tourist-attraction discovery through OpenStreetMap data
- Route-corridor filtering
- Detour distance and time estimation
- Basic ranking by detour and category match
- Interactive map with route and markers
- FastAPI endpoints and structured JSON responses

Deliberately excluded from MVP

- Hotel booking and payment
- Restaurant planning
- Budget optimization
- Voice assistant
- Full multi-day itinerary generation
- Behavior-learning recommendation model
- Multi-agent orchestration
- Mobile application

MVP user journey

Step	User/system action
1	User enters source, destination, interests, and maximum detour.
2	System geocodes the locations and requests the base route.
3	System builds a geographic corridor around the route.
4	System retrieves and filters tourist attractions in that corridor.
5	System estimates detours and removes unsuitable

Step	User/system action
	candidates.
6	System ranks, returns, and plots the best stops.

4. High-Level Architecture

The architecture separates route generation, geospatial discovery, ranking, and presentation so each component can be tested independently and upgraded later.

Layer	Primary responsibility	Suggested technology
Client	Collect trip criteria and display map/results	React + Leaflet
API	Validation, orchestration, response models	FastAPI + Pydantic
Routing	Generate paths, distances, and travel times	OSRM
Discovery	Retrieve tourist POIs in the route corridor	Overpass API + OpenStreetMap
Geospatial	Buffers, distances, intersections, spatial indexes	PostgreSQL + PostGIS
Ranking	Apply rule-based and later AI-based scoring	Python service
Cache	Reduce repeated route and place queries	Redis
AI layer	Preference extraction and explanation	Ollama with an open model

Logical data flow

7. Trip request
8. Geocoded coordinates
9. Base route polyline
10. Buffered route corridor
11. Candidate attractions
12. Filtered and enriched candidates
13. Ranked recommendations
14. Map-ready response

ARCHITECTURE NOTE
The LLM should not calculate geographic distances. Deterministic routing and spatial services should produce distances and times; AI should interpret preferences, rank contextually, and explain recommendations.

5. Open-Source Technology Stack

Component	Choice	Purpose
Backend	FastAPI	REST endpoints, orchestration, validation, async integration
Data models	Pydantic	Typed request and response contracts
Database	PostgreSQL	Trips, users, attractions, feedback
Spatial extension	PostGIS	Route buffers, proximity checks, spatial indexing
Map data	OpenStreetMap	Road and point-of-interest data
Routing engine	OSRM	Route geometry, distance, duration, waypoint routing
POI discovery	Overpass API	Query tourism and attraction tags
Map UI	Leaflet	Interactive route and marker rendering
Frontend	React	Trip form, results cards, map controls
Cache	Redis	Cache geocoding, routes, and repeated POI queries
Local AI runtime	Ollama	Run open models locally during development
Workflow layer	LangGraph, later	Stateful orchestration when the flow becomes complex

Important implementation rule

Keep third-party services behind adapter interfaces. For example, RouteProvider, PlaceProvider, Geocoder, and PreferenceRanker can each have a replaceable implementation. This reduces vendor lock-in and makes testing easier.

6. Route-Aware Attraction Discovery

Near the route vs. along the route

A useful recommendation is not simply close to some point in the region. It should lie within a configurable corridor around the actual route and should require an acceptable deviation from the original trip.

Stage	Processing logic
Route geometry	Decode the route polyline into geographic coordinates.
Corridor creation	Create a buffer around the route, such as 5 km or 10 km.
Candidate retrieval	Query tourism, viewpoint, museum, attraction, artwork, historic, cave, waterfall, and related tags.
Spatial filtering	Keep only candidates inside or close to the corridor.
Detour calculation	Compare the original route with a route containing the candidate stop.
Constraint filtering	Remove stops exceeding maximum extra distance or time.
Deduplication	Merge duplicate or near-identical POIs.

Detour metrics

- Distance from route: shortest geographic distance from the POI to the route geometry.
- Added route distance: distance with the stop minus original route distance.
- Added route time: duration with the stop minus original route duration.
- Direction suitability: penalize stops that require backtracking or crossing to the opposite travel direction.

QUALITY RULE
Show both proximity and real detour. A place can be geographically close to the route but still require a long road deviation.

7. Ranking and Recommendation Logic

Start with a transparent weighted score. Later, replace or augment selected terms with AI or ML without changing the overall API contract.

Factor	Meaning	Example weight
Interest match	Similarity between user interests and POI category/metadata	35%
Added time	Lower detour receives a higher score	30%
Added distance	Shorter deviation receives a higher score	15%
Data quality	Name, category, description, opening data completeness	10%
Diversity	Avoid recommending many identical attraction types	10%

Illustrative scoring formula

FINAL SCORE
0.35 × interest match + 0.30 × time score + 0.15 × distance score + 0.10 × data quality + 0.10 × diversity

Recommended output fields

- place_id, name, latitude, longitude
- category and matched user interest
- distance_from_route_km
- added_distance_km and added_time_minutes
- score and short explanation
- map marker data and optional image/reference metadata

Guardrails

- Never invent ratings when the data source does not provide them.
- Label missing information explicitly.

- Use deterministic calculations for route metrics.
- Explain ranking using the actual score components.

8. API and Data Design

Suggested API endpoints

Method	Endpoint	Purpose
POST	/routes	Generate the base route.
POST	/discover	Find route-corridor attractions.
POST	/recommend	Rank discovered attractions.
POST	/routes/optimize	Generate a route with selected stops.
POST	/feedback	Record clicks, saves, skips, and visits.
GET	/trips/{trip_id}	Retrieve a saved trip and recommendations.

Core database entities

Entity	Important fields
users	user_id, optional profile fields, preference settings
trips	trip_id, source, destination, route geometry, base distance, base time
attractions	place_id, name, category, coordinates, source tags
trip_candidates	trip_id, place_id, distance from route, detour metrics, score
feedback	user_id, trip_id, place_id, action, timestamp

Input contract example

REQUEST FIELDS
source, destination, interests[], maximum_detour_km, maximum_extra_minutes,

transport_mode, maximum_results

Output contract example

RESPONSE FIELDS
trip_id, base_route, recommended_stops[], totals, warnings, map_bounds, route_provider_metadata

9. AI and Multi-Agent Evolution

Phase 2: AI preference understanding

Allow the user to write natural language such as: “I like waterfalls, scenic viewpoints, and photography, but I do not want more than 30 minutes of extra driving.” The AI layer converts the text into structured preferences and constraints.

Phase 3: stateful workflow

Agent or node	Responsibility	Deterministic tools
Route node	Obtain and validate the base route	Geocoder, OSRM client
Discovery node	Find candidate attractions	Overpass client, PostGIS queries
Enrichment node	Normalize and enrich POI metadata	Database, metadata adapters
Ranking node	Score preference fit and explain choices	Rule engine, optional LLM
Planning node	Select compatible stops and construct final plan	Routing client, constraint solver
Validation node	Check constraints and unsupported claims	Schema and metric checks

WHEN TO USE LANGGRAPH
Use a workflow framework only after the basic pipeline works. It becomes valuable when there are retries, branching, persistent state, human selection, or multiple tool calls that must be coordinated.

Agent input/output discipline

Every agent or workflow node should use a typed input/output contract. This avoids ambiguous handoffs and makes the system easier to test, monitor, and debug.

10. Machine Learning Personalization

Machine learning should be introduced only after the product collects meaningful user feedback. Before that point, transparent content-based ranking is more reliable than training on insufficient data.

Useful feedback signals

- Attraction opened
- Attraction saved
- Stop added to route
- Stop removed
- Recommendation skipped
- Trip completed
- Explicit like/dislike
- Time spent viewing details

Model progression

Stage	Approach	Why
1	Rule-based scoring	Works without training data and is explainable.
2	Content-based recommendation	Matches POI features with explicit and inferred interests.
3	Learning-to-rank model	Learns which candidates users select under trip constraints.
4	Hybrid recommender	Combines content, behavior, context, and popularity.

Potential features

- Attraction category and tags
- Route segment and trip length
- Added time and distance
- Travel season and time of day
- User preference vector
- Past selections and skips
- Group type and accessibility constraints, when explicitly supplied

PRIVACY PRINCIPLE

Collect only data needed for recommendation quality. Provide clear controls for saving history and deleting personalization data.

11. Practical Implementation Roadmap

Phase	Deliverable	Key tasks
1. Foundation	FastAPI project and provider adapters	Schemas, configuration, logging, tests, error handling
2. Route core	Source-to-destination route	Geocoding, OSRM integration, polyline storage
3. Discovery	Attractions in route corridor	Overpass queries, buffers, PostGIS filtering
4. Detours	Accurate extra distance and time	Candidate waypoint routing, limits, caching
5. Ranking	Top N recommendations	Scoring, diversity, explanations
6. Map UI	Interactive working demo	React form, Leaflet route, markers, result cards
7. AI layer	Natural-language preference parsing	Local model, structured output, evaluation
8. Production polish	Deployable portfolio system	Docker, monitoring, tests, documentation

Suggested first milestone

MILESTONE 1
Given two coordinates, return the OSRM route geometry, distance, and duration through a tested FastAPI endpoint.

Suggested second milestone

MILESTONE 2
Given the route geometry, return at least five correctly tagged attractions within a configurable corridor and plot them on a map.

Definition of done for MVP

- End-to-end demo works using real route and POI data.
- Recommendations show calculated detour metrics.
- Failures are handled with readable messages.
- At least unit, integration, and one end-to-end test are included.
- README explains architecture, setup, trade-offs, and limitations.

12. Engineering Challenges and Testing

Challenge	Mitigation
Incomplete POI metadata	Normalize tags, expose missing fields, and avoid unsupported claims.
Large or complex routes	Sample intelligently, use spatial indexes, paginate and cache queries.
Overpass rate limits	Cache results, use bounded areas, add retries and respectful backoff.
Duplicate attractions	Deduplicate using source IDs, coordinates, and normalized names.
Slow detour calculation	Pre-filter geometrically before expensive waypoint routing.
No valid attractions	Return an empty result with suggestions to widen the corridor or interests.
LLM inconsistency	Require structured output and validate against allowed categories and bounds.

Testing strategy

- Unit tests for scoring, normalization, and constraints
- Contract tests for provider adapters
- Integration tests for database spatial queries
- Recorded responses for external-service test stability
- End-to-end test from trip input to map-ready response
- Evaluation set for preference extraction and ranking quality
- Performance tests for long routes and many candidates

Observability

- Request latency by stage
- Number of discovered and filtered candidates
- Cache hit rate
- External API failures and retries
- Average detour calculation cost
- Recommendation selection and dismissal rate

13. Portfolio and Interview Positioning

Resume-ready project description

RESUME BULLET
Built an AI Route Explorer that discovers and ranks tourist attractions along road-trip routes using geospatial analysis, route-corridor filtering, detour optimization, and preference-based recommendation. Developed a FastAPI backend with OpenStreetMap, OSRM, PostgreSQL/PostGIS, Redis, and an optional local LLM workflow for structured preference extraction and recommendation explanations.

Skills demonstrated

Area	Evidence in project
Backend engineering	Typed APIs, adapters, validation, caching, asynchronous integration
Geospatial systems	Polylines, buffers, proximity, spatial indexing, map visualization
Algorithms	Filtering, weighted ranking, constraint handling, route comparison
AI engineering	Structured LLM output, tool boundaries, workflow orchestration
Machine learning	Feedback design, content-based ranking, future learning-to-rank
Production thinking	Testing, monitoring, error handling, modular architecture

Interview discussion points

- Why route proximity differs from real detour
- Why the LLM should not perform distance calculations
- How spatial indexing improves candidate filtering
- How caching reduces routing and discovery latency
- How ranking evolves from rules to machine learning
- How provider adapters prevent lock-in

- How you would evaluate recommendation usefulness

14. Future Product Scope

Version	Expansion
V2	AI-based interest parsing and recommendation explanations
V3	Stateful LangGraph workflow with retries and user stop selection
V4	Personalized recommendations from user feedback
V5	Day-wise itinerary generation and opening-time awareness
V6	Hotels, restaurants, and rest stops as optional categories
V7	Budget, fuel, accessibility, and group-travel constraints
V8	Voice interface and mobile-friendly experience

Final recommendation

Treat AI Route Explorer as a staged engineering project. Complete the route, discovery, and detour pipeline first. Then add AI for natural-language preference extraction and explanations. Add multi-agent orchestration only when the workflow has enough branching and state to justify it. Add ML personalization only after collecting real signals.

FLAGSHIP PROJECT POTENTIAL
A polished end-to-end implementation can become a strong flagship portfolio project because it demonstrates AI integration, geospatial reasoning, recommendation design, backend engineering, and production trade-offs in one practical system.

Immediate next actions

15. Create the FastAPI repository and typed request/response models.
16. Run or connect to an OSRM routing service.
17. Build the first /routes endpoint.
18. Query one attraction category through Overpass.
19. Plot the route and POIs using Leaflet.
20. Add route-corridor filtering and detour calculation.
21. Document every milestone with screenshots, tests, and decisions.